

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчёт по лабораторной работе
«Вычисление арифметических выражений»

Выполнил:

Каргин Никита Александрович
3824Б1ПМ4

Проверила:

Бусько П.В.

Нижний Новгород
2025

Оглавление

Введение	3
Цели и задачи	4
Используемые инструменты и общая структура проекта	5
Руководство пользователя.....	6
Описание программной реализации класса TStack	8
Описание программной реализации класса TArifmeticExpression.....	11
Заключение	15
Список используемой литературы.....	16
Приложение 1.	17

Введение

Постфиксная запись, также известная как обратная польская запись (RPN), представляет собой способ записи математических и логических выражений, где операторы располагаются после операндов, которые они обрабатывают. Постфиксная нотация – это не просто интересная математическая концепция, а мощный инструмент с реальными преимуществами, особенно в области компьютерных наук:

Оптимизация вычислений и анализа выражений: Это основное достоинство постфиксной записи. Вместо сложной обработки инфиксных выражений, требующей учета приоритета операций, ассоциативности и скобок (что реализуется, например, алгоритмом сортировочной станции Дейкстры), постфиксная запись значительно упрощает задачу. Она представляет выражение в форме, идеально подходящей для непосредственного вычисления. Простой и эффективный алгоритм, использующий стек, позволяет вычислить значение постфиксного выражения за один проход. По этой причине компиляторы часто преобразуют инфиксные выражения во внутреннее представление, такое как постфиксная запись или дерево выражений, для упрощения последующей генерации машинного кода.

Удобство для стековых и виртуальных машин: Архитектура многих виртуальных машин, таких как JVM или Forth, основана на стеке. Инструкции в их байт-коде часто напрямую соответствуют принципам постфиксной записи, что делает ее естественным и эффективным способом представления операций.

Легендарные инженерные калькуляторы от Hewlett-Packard (HP) долгое время использовали постфиксную запись (RPN). Многие инженеры и ученые до сих пор предпочитают их из-за эффективности: не нужно использовать скобки, а последовательность нажатий становится очень логичной и требует меньше действий для сложных вычислений.

Цели и задачи

Цель данной работы — разработка структуры данных Стек и ее использование для расчета арифметических выражений с использованием обратной польской записи (постфиксной формы).

Необходимо разработать программу, выполняющую вычисление арифметического выражения с вещественными числами. Выражение в качестве операндов может содержать переменные и вещественные числа. Допустимые операции известны: $+$, $-$, $/$, $*$. Допускается наличие знака "-" в начале выражения или после открывающей скобки. Опционально - наличие математических функций ($\sin$, $\cos$, $\ln$, $\exp$, и т.д.) Программа должна выполнять предварительную проверку корректности выражения и сообщать пользователю вид ошибки и номера символов строки, в которых были найдены ошибки.

Выполнение работы предполагает решение следующих задач:

- Разработка интерфейса шаблонного класса TStack.
- Реализация методов шаблонного класса TStack.
- Разработка интерфейса класса TPostfix для работы с постфиксной формой.
- Реализация методов класса TPostfix.
- Разработка и реализация тестов для классов TStack и TPostfix на базе Google Test.
- Публикация исходных кодов в личном репозитории на GitHub.

Используемые инструменты и общая структура проекта

Система контроля версий Git.

Фреймворк для написания автоматических тестов Google Test. Не требует установки, идет вместе с проектом-шаблоном.

Среда разработки Microsoft Visual Studio 2022

Структура проекта:

- gtest — библиотека Google Test.
- samples — каталог с пользовательским приложением.
- test — каталог с проектом с модульными тестами.
- include src - каталоги с основными файлами ЛР.
- sln - каталог с файлами с решениями (solution) для Microsoft Visual Studio 2010 и 2012.
- README.md — информация о проекте, которую вы сейчас читаете.

Служебные файлы

.gitignore — перечень расширений файлов, игнорируемых Git при добавлении файлов в репозиторий.

Руководство пользователя

После запуска программы вы увидите меню с четырьмя пунктами. Вам нужно последовательно выполнять действия, чтобы получить результат вычислений.

```
1. Enter a new expression
2. Convert to postfix form
3. Enter variable values
4. Evaluate the expression
Select action: |
```

Рис. 1 – Вид консольного приложения

Шаг 1: Ввод выражения

Сначала выберите пункт "Ввести новое выражение". Введите математическое выражение, используя:

Переменные: строчные английские буквы (a, b, c, ... z)

Операции: + (сложение), - (вычитание), * (умножение), / (деление), ^ (степень)

Скобки: () для изменения порядка вычислений

Если вы введете пустое выражение, программа сообщит об ошибке.

Шаг 2: Преобразование выражения

После успешного ввода выражения выберите пункт "Преобразовать в постфиксную форму". Программа переведет ваше выражение в специальный формат, который упрощает вычисления.

На этом этапе программа также проверяет корректность выражения: правильность скобок, допустимость символов.

Если преобразование прошло успешно, вы увидите выражение в новой форме. Например, выражение $(a+b)*c$ превратится в $ab+c*$.

Шаг 3: Задание значений переменных

```
To exit, press 0
Enter a variable name (one letter) or 0 to exit:|
```

Рис. 2 – Ввод значений

Теперь нужно указать, какие числа соответствуют переменным в вашем выражении. Выберите пункт "Ввести значения переменных".

Для каждой переменной:

Введите имя переменной (одну букву)

Введите ее числовое значение

Повторите для всех переменных, которые есть в выражении

Для завершения ввода введите 0

Если вы попытаетесь задать значение для символа, который не является строчной английской буквой, программа сообщит об ошибке. Если в выражении есть переменные, которые вы не задали, при вычислении они будут считаться равными нулю.

Шаг 4: Вычисление результата

Когда все готово, выберите пункт "Вычислить выражение". Программа подставит заданные значения переменных, выполнит вычисления и покажет результат.

Если вы попытаетесь вычислить выражение до его преобразования в постфиксную форму, программа напомнит, что нужно сначала выполнить второй шаг.

Описание программной реализации класса TStack

Класс TStack реализует классическую структуру данных "стек" по принципу LIFO (Last-In-First-Out) с использованием динамического массива. Основная идея - организация данных, где элементы добавляются и извлекаются только с одного конца (вершины стека).

При создании стека выделяется фиксированный массив заданного размера. Память распределяется один раз в конструкторе и освобождается в деструкторе. Это обеспечивает предсказуемость производительности, но ограничивает максимальный размер.

Описание private полей класса:

1. `int top`. Целочисленная переменная, которая служит индексом или указателем на вершину стека.

Основная задача: Определяет позицию, куда будет помещен следующий элемент (при push) и откуда будет взят элемент при извлечении (при pop).

Контроль состояния: Значение `top` напрямую указывает на текущее состояние стека:

`top == -1`: Стек пуст (классическая начальная инициализация).

`top == sizeStack - 1`: Стек полностью заполнен.

Упрощение логики: Благодаря этой переменной операции push и pop выполняются за время $O(1)$, так как не требуется перемещать другие элементы.

2. `int sizeStack`. Целочисленная переменная, хранящая максимальную вместимость стека (размер внутреннего массива `data`).

Основная задача: Управление памятью. Фиксирует, сколько элементов типа T было выделено в динамической памяти.

Контроль переполнения: Позволяет в методе push проверить условие `if (top == sizeStack - 1)`, чтобы предотвратить запись за пределами выделенной памяти (что привело бы к неопределенному поведению или падению программы).

Необходима для корректной работы: Используется в конструкторе (для выделения памяти), в проверках границ и в деструкторе (для освобождения правильного объема памяти).

3. `T* data`. Указатель на тип T. Это указатель на начало динамически выделенного массива, который и является физическим хранилищем данных стека.

Основная задача: Хранение элементов. Непосредственно содержит элементы стека, расположенные в памяти последовательно.

Гибкость и эффективность: Динамическое выделение памяти позволяет создавать стеки произвольного размера, задаваемого во время выполнения программы, в отличие от стека на статическом массиве.

Основа реализации: Все операции стека (push, pop, peek) работают с этой областью памяти, обращаясь к элементу по индексу top.

Операции:

push: top++ затем запись в data[top]

```
void push(const T& value) {  
    if (top >= sizeStack - 1) {  
        throw std::string("stack is full");  
    }  
    data[++top] = value;  
}
```

Рис. 3 - Програмная реализация метода push

pop: чтение data[top] затем top --

```
T pop() {  
    if (isEmpty()) {  
        throw std::string("Stack is empty");  
    }  
    return data[top--];  
}
```

Рис. 4 - Програмная реализация метода pop

seeLast: чтение data[top] без изменения размеров стека

```
T& seeLast() {  
    if (isEmpty()) {  
        throw std::string("Stack is empty");  
    }  
    return data[top];  
}
```

Рис. 5 - Програмная реализация метода seeLast

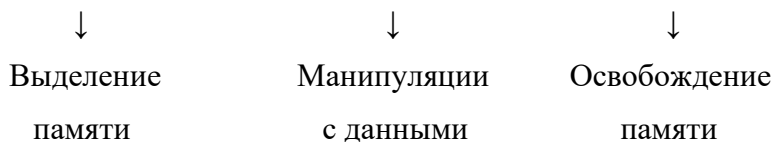
Преимущества шаблонного подхода:

- Повторное использование кода - один класс для всех типов
- Статическая типобезопасность - ошибки типов обнаруживаются на этапе компиляции
- Отсутствие накладных расходов - специализация для каждого типа происходит на этапе компиляции

Используется стратегия "fail-fast" через исключения. Некорректная инициализация - отрицательный размер стека. Переполнение - попытка добавить в полный стек. Опустошение - попытка извлечь из пустого стека. Это предотвращает неопределенное поведение и делает код более надежным.

Жизненный цикл стека:

Создание → Использование (push/pop) → Уничтожение



Все операции: $O(1)$ - постоянное время. Нет циклов или рекурсии в методах.

Пространственная сложность: $O(n)$ - где n максимальный размер стека

Преимущества:

- Высокая производительность операций
- Простота реализации и использования
- Типобезопасность благодаря шаблонам
- Предсказуемое использование памяти

Описание программной реализации класса

TArifmeticExpression

Класс TArifmeticExpression предназначен для работы с арифметическими выражениями:

Хранение выражений в инфиксной и постфиксной (обратной польской) нотации, Преобразование между нотациями, Вычисление значения выражения, Работа с переменными (операндами)

Описание private полей класса:

1. `std::string infix`. Строка, хранящая исходное выражение в инфиксной (обычной) нотации.

Основное хранилище: Содержит входные данные, переданные пользователем.

Восстановление: Позволяет получить исходное выражение через метод `getInfix()`.

Повторная обработка: При изменении выражения через `setExpression()` обновляется это поле.

Валидация: Проверяется на пустоту в конструкторе и сеттере.

2. `std::string postfix`

Основная задача: Строка, хранящая выражение в обратной польской (постфиксной) нотации.

Результат преобразования: Заполняется после успешного выполнения `toPostfix()`.

Ускорение вычислений: Постфиксная нотация не требует скобок и упрощает алгоритм вычисления.

Промежуточное представление: Используется методом `calculate()` для вычисления значения.

Кэширование: После преобразования выражение можно вычислять многократно без повторного парсинга.

3. `std::vector<char> lexems`. Вектор символов, содержащий токенизированное представление выражения.

Основная задача: Лексический анализ. Хранит результат разбиения строки `infix` на отдельные лексемы (токены).

Упрощение обработки: Удалены пробелы, что упрощает дальнейший синтаксический анализ.

Промежуточное состояние: Используется только при преобразовании в постфиксную форму в `toPostfix()`.

Валидация символов: При заполнении проверяется корректность каждого символа.

4. `std::map<char, int> priority`. Ассоциативный массив, сопоставляющий операторам их приоритет.

Основная задача: Определение порядка операций: Ключевая структура для алгоритма преобразования инфикс→постфикс (алгоритм Дейкстры/сортировочной станции).

Гибкость настройки: Позволяет легко изменять приоритеты операций.

Обработка скобок: Скобки (и) также имеют приоритет (обычно 0 или 1), что важно для алгоритма.

Быстрый доступ: $O(\log n)$ доступ к приоритету оператора при сравнении.

5. `std::map<char, double> operands`. Ассоциативный массив, связывающий имена переменных с их значениями.

Основная задача: Хранение значений переменных. Позволяет задавать значения для переменных (например, $a=5$, $b=3.2$).

Подстановка при вычислении: В методе `calculate()` значения подставляются вместо переменных.

Поддержка множества переменных: Может хранить значения для всех переменных от 'a' до 'z'.

Проверка существования: При обращении к неинициализированной переменной вернется 0 (значение по умолчанию для `double`).

Важность использования стека

Метод `toPostfix()`: (см. Приложение 1)

Метод `toPostfix()` является ключевым в работе класса, он позволяет перевести инфиксную нотацию вводимую пользователем, в постфиксную, используя ранее реализованный Стэк

Алгоритм работы:

Операнды (переменные) сразу добавляются в выходную строку

Операторы помещаются в стек согласно приоритетам

Открывающая скобка всегда помещается в стек

Закрывающая скобка выталкивает операторы до открывающей скобки

Операторы выталкиваются из стека когда встречается оператор с меньшим приоритетом

Метод calculate():

```
double calculate() {
    if (postfix.empty()) {
        throw std::string("call toPostfix()");
    }
    TStack<double> stack;
    for (int i = 0; i < postfix.length(); i++) {
        char curEl = postfix[i];
        if (curEl >= 'a' && curEl <= 'z') {
            stack.push(operands[curEl]);
        }
        else {
            double a = stack.pop();
            double b = stack.pop();
            switch (curEl) {
                case '+': stack.push(a + b); break;
                case '-': stack.push(b - a); break;
                case '*': stack.push(a * b); break;
                case '/': stack.push(b / a); break;
            }
        }
    }
    return stack.pop();
}
```

Рис. 6 - Програмная реализация метода calculate()

Не менее важным методом класса является calculate(), представленный на рисунке 5, который считает значение выражения используя потфиксную запись и специальный алгоритм:

- Инициализация: Создаем пустой стек.
- Разбиение строки: Преобразуем входную строку в список элементов (токенов). Токены разделены пробелами.

Например, строка "5 1 2 + 4 * + 3 -" превратится в список ['5', '1', '2', '+', '4', '*', '+', '3', '-'].

Обход токенов: Проходим по каждому токену в списке слева направо.

Токен — число?

- Преобразуем его в числовой тип (например, int или float).
- Помещаем это число на вершину стека.

Токен — оператор (например, +, -, *, /)?

- Извлекаем из стека два верхних элемента. Важно: первый извлеченный элемент является правым операндом, второй — левым.

operand2 = stack.pop() (правый операнд)

operand1 = stack.pop() (левый операнд)

- Выполняем операцию: result = operand1 <оператор> operand2

Например, для +: result = operand1 + operand2

- Помещаем полученный результат result обратно в стек.

Завершение: После обработки всех токенов в стеке должен остаться ровно один элемент. Это и есть итоговый ответ. Извлекаем и возвращаем его.

Стек является ключевой структурой данных в этом классе по двум причинам:

1. В методе toPostfix():

Преобразование инфиксной нотации в постфиксную (алгоритм Дейкстры "Сортировочная станция")

Стек временно хранит операторы в правильном порядке приоритетов

Обеспечивает корректную обработку скобок

2. В методе calculate():

Вычисление значения выражения в постфиксной нотации

Стек хранит промежуточные результаты вычислений

Позволяет последовательно применять операции к операндам

Заключение

Достигнутые результаты

Разработан шаблонный класс TStack, реализующий классическую структуру данных "стек" с принципом LIFO (Last-In-First-Out). Класс демонстрирует высокую эффективность операций (все методы работают за время $O(1)$) и обеспечивает типобезопасность благодаря использованию шаблонов C++.

Создан класс TArifmeticExpression для работы с арифметическими выражениями, который:

Выполняет преобразование инфиксной записи в постфиксную с использованием алгоритма Дейкстры. Осуществляет вычисление выражений в постфиксной форме. Поддерживает работу с переменными и базовыми арифметическими операциями

Практически доказана важность стека как фундаментальной структуры данных в компьютерных науках. Стек оказался незаменимым инструментом для:

- Управления приоритетами операций при преобразовании нотаций
- Хранения промежуточных результатов при вычислениях
- Обеспечения корректной обработки скобок

Реализована система тестирования на базе Google Test, обеспечивающая надежность и корректность работы разработанных классов.

Практическая значимость

Выполненная работа имеет важное практическое значение, поскольку продемонстрировала реальные сценарии применения стека в таких областях, как:

- Вычислительные системы и инженерные калькуляторы
- Алгоритмы обработки математических выражений

Проведенная работа не только позволила глубоко понять принципы работы стека и постфиксной нотации, но и продемонстрировала их фундаментальную важность в компьютерных науках и практическом программировании. Полученные знания и навыки могут быть успешно применены при решении широкого круга задач, связанных с обработкой данных и разработкой алгоритмов

Список используемой литературы

1. **Никлаус Вирт.**
Алгоритмы и структуры данных. / Пер. с англ. Ткачев Ф. В. — М.: ДМК Пресс, 2010. — 272 с.
2. **Страуструп, Бьерн.**
Язык **программирования С++** [Текст] / Бьерн Страуструп ; пер. с англ. под ред. Н. Н. Мартынова. - Спец. изд. - Москва : Бином, 2011. - 1135 с. : ил.; 24 см.; ISBN 978-5-7989-0425-9
3. **Язык программирования С#** [Текст] / А. Хейлсберг [и др.] ; [пер. с англ. Р. Тетерин]. - 4-е изд. - Москва [и др.] : Питер, 2012. - 773 с. : ил.; 24 см. - (Классика Computer Science).; ISBN 978-5-459-00283-6
4. **Сластихина, М. Д.**
Алгоритмы и структуры данных [Электронный ресурс] : учебное пособие по курсу "Алгоритмы и структуры данных" для студентов направления "Программная инженерия" 231000.62 : электронное издание локального распространения / М. Д. Сластихина, Т. Э. Шульга, О. А. Торопова ; М-во образования и науки Российской Федерации, Саратовский гос. технический ун-т им. Гагарина Ю. А. - Саратов : СГТУ им. Гагарина Ю. А., 2015. - 1 электрон. опт. диск (CD-ROM); 12 см.; ISBN 987-5-7433-2860-4

Приложение 1.

Програмная реализация метода toPostfix()

```
void toPostfix() {
    toLexems();
    std::string variables = "";
    TStack<char> stack;
    bool openingBracket = false;
    for (int i = 0; i < size(lexems); ++i) {
        const char& lexem = lexems[i];
        if (lexems[i] >= 'a' && lexems[i] <= 'z') {
            variables += lexems[i];
        }
        else {
            size_t stack_prior;
            if (!stack.isEmpty()) stack_prior =
priority[stack.seeLast()];
            else stack_prior = 0;
            size_t cur_prior = priority[lexems[i]];
            if (lexems[i] == '(') {
                stack.push(lexems[i]);
                openingBracket = true;
            }
            else if (lexems[i] == ')') {
                if (openingBracket) {
                    while (stack.seeLast() != '(' &&
!stack.isEmpty()) {
                        variables += stack.pop();
                    }
                    if (!stack.isEmpty() && stack.seeLast() ==
'(') {
                        stack.pop();
                    }
                    openingBracket = false;
                }
                else {
                    throw std::string("not found '('");
                }
            }
            else if (cur_prior >= stack_prior) {
                stack.push(lexems[i]);
            }
            else if (cur_prior < stack_prior) {
                while (!stack.isEmpty() &&
priority[stack.seeLast()] >= cur_prior) {
                    variables += stack.pop();
                }
                //variables += stack.pop();
                stack.push(lexems[i]);
            }
        }
    }
}
```

```

        else {
            throw std::string("Anknown simb");
        }
    }
}
while (!stack.isEmpty()) {
    variables += stack.pop();
}
std::cout << variables;
postfix = variables;
}

```